Cquirrel: Continuous Query Processing over Acyclic Relational Schemas

Student ID: 21185592
Student name: Chai Guanpeng

Abstract

This project reproduces the three-table Q3 example in Cquirrel using the authors' code generator and core implementation on Flink under Windows and WSL2. Correctness is checked with 22 small scenarios and six TPC-H scenarios, covering insertion, deletion, restoration, late data and predicate boundaries. A further 32 runs examine two data scales, two workloads and two parallelism settings. All jobs pass final-result validation against independent SQL answers. Increasing parallelism from one to two does not reduce median job duration in this small, single-machine experiment. The deliverables include executable query code, repeatable experiments, raw evidence and a report of the observed limitations.

Keywords: Cquirrel; incremental view maintenance; multiway joins; Flink; TPC-H

1. Background of the project and related work

1.1 Motivation and reproduction objectives

When customer, order and line-item records change continuously, revenue statistics must also change. Recomputing a complete join and aggregation after every update can repeatedly scan unchanged data. Incremental view maintenance retains state and processes the affected portion. Cquirrel targets queries whose foreign-key relationships form a directed acyclic graph and whose joins equate primary and foreign keys; it compiles maintenance logic into Flink programs [1].

This simple reproduction asks whether the official implementation correctly maintains the final Q3 answer in the available environment, and how job duration changes with parallelism at scale factors 0.001 and 0.01. It reuses the official algorithm and adds data generation, execution and validation tools. Cross-system rankings, failure recovery and reproduction of every paper figure are outside this scope.

1.2 Related work

DBToaster recursively derives higher-order deltas and materializes auxiliary views to reduce maintenance cost [3]. Trill combines a temporal relational model with batched columnar representations and dynamic compilation for diverse analytical workloads [4]. These systems also address changing data, but use different query models and execution approaches. They were not deployed as performance baselines here; the measurements compare configurations of the same Cquirrel program.

2. Description of the query and the algorithm

2.1 Query semantics

The query is the Q3 variant in Section 2 of the paper. It selects AUTOMOBILE customers, orders placed before 1995-03-13 and line items shipped after that date. Discounted revenue is grouped by order key, order date and shipping priority. There is no ORDER BY or LIMIT: every qualifying group is returned, rather than the full standard TPC-H Q3 Top-10 output.

```
SELECT l_orderkey, o_orderdate, o_shippriority,
       sum(l_extendedprice * (1 - l_discount)) AS revenue
FROM customer, orders, lineitem
WHERE c_mktsegment = 'AUTOMOBILE'
  AND c_custkey = o_custkey
  AND l_orderkey = o_orderkey
  AND o_orderdate < DATE '1995-03-13'
  AND l_shipdate > DATE '1995-03-13'
GROUP BY l_orderkey, o_orderdate, o_shippriority;
```

2.2 Join direction and live-state conditions

The foreign-key direction is Lineitem -> Orders -> Customer, while update propagation runs in the opposite direction. For this chain, a record is live when it exists, passes its local predicate and can join to the required parent. The conditions are recursive:

A customer is live if it exists and belongs to AUTOMOBILE.
An order is live if it exists, predates the boundary and has a live customer.
A line item is live if it exists, ships after the boundary and has a live order.

Receiving a line item therefore does not immediately imply a revenue contribution. If its order or customer has not arrived, it cannot yet form a complete join. It can enter the result later when its parent becomes live. This explanation applies to the Q3 chain; a general DAG with converging paths additionally requires consistency between matches along those paths [1].

Customer -> Orders -> Lineitem -> Aggregate -> File sink

Figure 1. Update propagation in the generated Q3 program.

2.3 Maintained state and indexes

Logically, the order layer needs a customer-key-to-orders index, while the line-item layer needs an order-key-to-line-items index. The operators also retain parent liveness and the attributes needed by the join. Records that pass local predicates but lack a live parent remain available for later activation. The aggregation layer stores current revenue for each order.

In the generated Job.scala, customer and order streams meet on the customer key. Order outputs and line-item inputs then meet on the order key, which also partitions aggregation [2]. Related records and updates therefore reach the same keyed state. A parent change visits its indexed dependants without scanning unrelated customers or orders.

2.4 Insertion and deletion propagation

Consider a newly inserted customer. The Q3 maintenance logic can be explained in four steps. In the actual program, separate Flink operators exchange messages asynchronously; a single function does not execute the entire sequence.

First, test the AUTOMOBILE predicate and update the customer key's live state. Second, locate stored orders under that key. Orders passing the date predicate become joinable and notify the next operator with their order keys, dates and priorities. Third, locate the corresponding line items and emit join additions for those passing the shipment-date predicate. Fourth, calculate their revenue contributions and update each affected order aggregate.

A line item arriving before its order is retained until that order becomes live. When the parent is already live, a new qualifying order or line item can propagate immediately. A record failing its local predicate contributes nothing to the query.

Deletion retracts previous contributions. Deleting a live line item removes only its contribution. Deleting a live customer invalidates the associated orders and their line items, generating removals downstream. This does not delete the original order or line-item records from their input relations: they can remain waiting and be reconsidered when the parent is restored.

2.5 Updating the aggregate

Let g=(order key, order date, shipping priority), let A[g] be its current revenue, and let w(l)=l_extendedprice*(1-l_discount). A join addition has sign s=+1 and a removal has s=-1. Each join change performs:

$$A[g] \leftarrow A[g] + s * w(l)$$

QAggregate0ProcessFunction supplies the contribution function, addition and subtraction to the core aggregate operator, which stores A[g] in ValueState. This version emits the updated A[g], not s*w(l). Consequently, summing every amount in the output file would be incorrect. Section 3 explains empty groups and floating-point tolerance.

2.6 Worked example, cost and validation target

Suppose order 101 has two qualifying contributions, 900 and 400. Insertion gives 0+900+400=1300. Deleting the first line item gives 1300-900=400. Deleting the customer retracts the remaining 400 and removes the order from the query answer. Restoring the customer reactivates the remaining contribution of 400. The six tested lifecycle prefixes therefore expect: no group -> 900 -> 1300 -> 400 -> no group -> 400.

Indexes restrict work to affected keys, avoiding complete join recomputation. However, deleting a customer with many dependent orders and line items can still produce many retractions; total processing and output cost is not always O(1). Each emitted join change requires one arithmetic update to SUM.

Each lifecycle prefix is replayed independently from empty state and checked after completion. This validates the resulting states without requiring every asynchronous output to follow input-event order. Additional insertion, deletion, late-parent and restoration cases check these transitions; Section 5 reports the coverage and results.

3. Implementation details

3.1 Official implementation and project additions

The reproduction pins Cquirrel-release commit 501cfde6dc2968cfccee992ab4f298c499cc8028 [2]. The pipeline is SQL -> JSON plan -> Scala operators -> Maven JAR -> Flink job. Relation and aggregation maintenance use the authors' implementation; this project adds repeatable execution, datasets and independent validation.

Table 1. Main implementation artifacts; see FILES.md for the full inventory.

Artifact	Role in this reproduction
queries/q3.sql; q3.generated.json	Define query semantics and retain the official parser output.
generated/.../Job.scala	Connect operators; adapt input/output arguments and text formatting.
Qcustomer / Qorders / Qlineitem / QAggregate0ProcessFunction.scala	Generated relation and aggregate operators using the original core JAR.
scripts/experiment.py	Generate scenarios, replay updates and compare with SQL answers.
scripts/benchmark.py; configs/benchmark.json	Schedule scales, warm-ups and repetitions; validate results and summarize time.
patches/job-cli-output.patch	Expose the changes to the generated job entry point.

3.2 Compatibility and output semantics

The official SQL parser matches aggregate names case-sensitively. Uppercase SUM omits aggregation metadata; code generation can print an exception yet return a successful exit code. The reproduction uses the equivalent lowercase sum and checks the plan, generated files and JAR explicitly. The failed attempt remains archived.

The aggregate emits updated group revenue, rather than an additive delta. The adapted format is order key|date|priority|revenue|source event number. Normalization retains the last arriving value per group. Source event numbers may be non-monotonic and must not reorder outputs. A single-parallelism file sink preserves arrival order.

After the last contribution is removed, the core may emit zero or a small floating-point residual, whereas SQL removes the group. All contributions in this experiment are strictly positive, so final absolute revenue at most 1e-6 is recorded as an empty-group sentinel. This convention does not generalize to valid zero revenue, negative contributions or cancellation in arbitrary SUM queries.

3.3 Independent correctness checks

Functional tests replay events into primary-key dictionaries, load final tables into DuckDB and execute the same SQL. Small cases additionally use hand-calculated answers. Timing cases query the generated database directly, excluding selected keys for deletion workloads. Group keys must match exactly; Cquirrel Double values are checked against DECIMAL reference amounts using:

$$|\text{actual} - \text{reference}| \leq \max(1e\text{-}6, 1e\text{-}9 * |\text{reference}|).$$

4. Experimental setup and data sets used

4.1 Execution environment

Table 2. Experimental environment.

Component	Configuration
System and hardware	Windows host; WSL2 / Ubuntu 20.04; Intel Core i7-14700HX; 28 logical CPUs and approximately 7.6 GiB visible RAM.
Build and runtime	OpenJDK 1.8.0_452; Maven 3.6.3; Flink 1.11.2 / Scala 2.12.
Scripts and generator	Python 3.8.10; DuckDB 1.1.3; signed official TPC-H extension.
Flink resources	One TaskManager, two slots; JobManager 1024 MB and TaskManager 1728 MB; operator parallelism p=1 or 2; file source/sink parallelism fixed at 1.

4.2 Datasets and update workloads

The DuckDB TPC-H extension generates synthetic data [5]; only Customer, Orders and Lineitem feed Q3. Generator versions, extension hashes, input hashes and deletion rules are archived. SF is a generation parameter; actual row counts below describe the input more precisely than an estimated file size.

Table 3. Source rows and workload event counts.

SF	Customers	Orders	Line items	Insertions	Deletions
0.001	150	1,500	6,005	7,655	601
0.01	1,500	15,000	60,175	76,675	6,018

The insert workload loads all customers, orders and line items in that order. The delete_10pct workload then removes every tenth line item in (orderkey, linenumber) order. Rounding makes the deletion share approximately 10%; total event counts are 8,256 and 82,693. These are deterministic deletions after insertion, not randomly interleaved updates.

4.3 Procedure and timing definition

First run 22 small scenarios and six SF=0.001 functional scenarios. Then evaluate 2 scales x 2 workloads x 2 parallelism settings: eight configurations, each with one warm-up and three measured runs. Within each measurement round, configuration order is shuffled using seed 20261023. All 32 runs must finish, apply the requested parallelism and pass answer validation; logs and raw outputs are retained.

The primary metric is Flink REST job duration, including deployment, input, operators and file output. It excludes data generation, compilation and the complete client startup overhead. Client wall-clock time is stored separately. Tables use the median and minimum/maximum of three measured runs, excluding warm-up. This is neither per-update latency nor sustained streaming throughput.

```
.\repro_final\run.ps1 all
.\repro_final\run.ps1 benchmark
```

5. Experimental results and discussion

5.1 Correctness

All 22 small scenarios, six TPC-H functional scenarios and 32 timing runs pass final-result validation: 60 verified Flink jobs in total. Small cases agree with hand calculations. Across the six first-stage TPC-H cases, the maximum absolute error for retained groups is 1e-11, within tolerance. Six verifier self-checks cover wrong amounts, missing/extra groups, floating-point tolerance, arrival order and empty-group sentinels.

In timing runs, both SF=0.001 workloads end with 11 groups. At SF=0.01, insertion ends with 107 and deletion with 101. Equal group counts do not imply unchanged revenue: values are also checked. Parent deletion and restoration cases support correctness of cross-table propagation within the tested coverage.

5.2 Job duration

Table 4. Three measured runs per row; speedup = T1 / Tp.

SF	Workload	p	Median/ms	Min-max/ms	Speedup
0.001	Insert	1	201	168-225	1.000
0.001	Insert	2	216	133-258	0.931
0.001	Insert + delete	1	169	168-174	1.000
0.001	Insert + delete	2	193	145-203	0.876
0.01	Insert	1	493	477-557	1.000
0.01	Insert	2	495	494-557	0.996
0.01	Insert + delete	1	549	502-619	1.000
0.01	Insert + delete	2	713	678-790	0.770

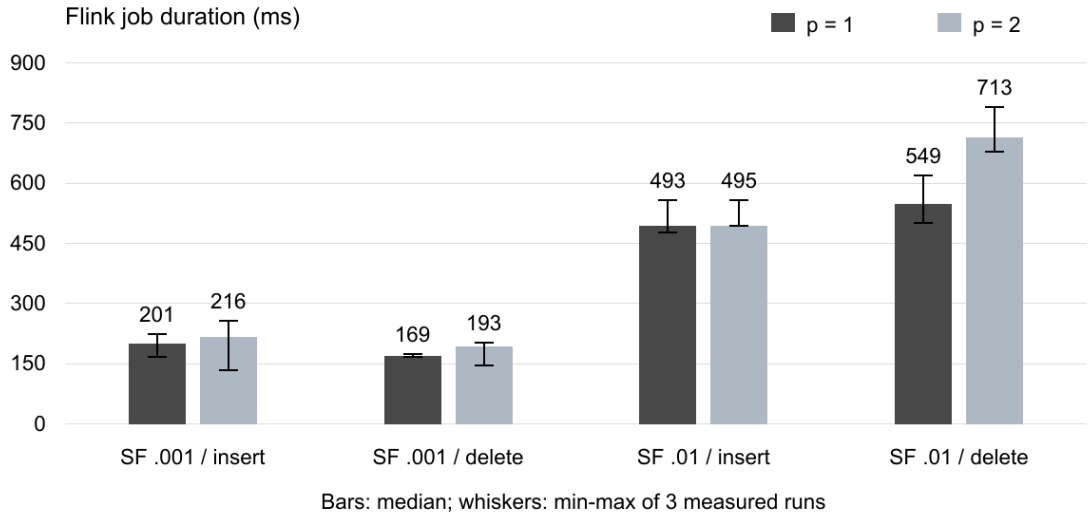


Figure 2. Bars show medians; whiskers show min/max, not confidence intervals.

5.3 Interpretation and limitations

At SF=0.01, increasing p from one to two changes median insertion time from 493 to 495 ms, and insertion-plus-deletion time from 549 to 713 ms. All four p=2 speedups are at most one: no parallel speedup is observed. At SF=0.001, jobs are shorter and run-to-run variation is larger relative to their duration.

Small jobs may be dominated by deployment, scheduling and file I/O overhead. Two slots on one TaskManager are not independent machines; the source and sink remain serial, and additional parallelism can add communication overhead. These are plausible explanations, not isolated causal findings: no component-level profiling was performed. Three repetitions per configuration are insufficient for a robust significance assessment.

Finite scenarios do not prove correctness in general. Empty-group handling assumes positive contributions. Bounded file replays do not measure continuous-stream latency, peak memory, checkpoint recovery or network failures. DuckDB is an answer oracle, not a timed competitor. The paper's scaling figure uses Q5, larger datasets and more workers [1]; this project uses Q3 with SF=0.001/0.01 and single-machine p=1/2. Results apply only to these conditions.

6. Future directions

First, extend to Q5, larger scale factors and multiple TaskManagers, with more repetitions and separate measurements of deployment, steady-state processing rate and update latency. Second, test randomly interleaved updates, skew, additional late-data cases and zero/negative contributions; maintain group counts or explicit retractions for general SUM semantics. Third, test checkpoints and recovery, and add other systems under matched semantics, hardware and timing definitions.

The reproduction now provides a repeatable path from query generation to execution and independent validation. Per-run logs, raw outputs and summaries support traceability. The English package preserves the same measured evidence as the Chinese edition; translating documentation does not represent additional experiments. File roles and rerun commands are listed in README.md and report/FILES.md.

7. The link of the project

https://github.com/Iamyg/independent-project-2

References

[1] Wang, Q., et al. Cquirrel: Continuous Query Processing over Acyclic Relational Schemas. PVLDB, 14(12): 2667-2670, 2021. DOI: 10.14778/3476311.3476315. https://www.cse.ust.hk/~yike/Cquirrel.pdf

[2] HKUST Database Group. Cquirrel-release. Commit 501cfde6dc2968cfccee992ab4f298c499cc8028. https://github.com/hkustDB/Cquirrel-release (accessed 2026-10-23).

[3] Ahmad, Y., Kennedy, O., Koch, C., Nikolic, M. DBToaster: Higher-order Delta Processing for Dynamic, Frequently Fresh Views. PVLDB, 5(10): 968-979, 2012. https://arxiv.org/abs/1207.0137

[4] Chandramouli, B., et al. Trill: A High-Performance Incremental Query Processor for Diverse Analytics. Microsoft Research / VLDB publication. https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/trill-vldb2015.pdf

[5] DuckDB. TPC-H Extension. https://duckdb.org/docs/stable/core_extensions/tpch (accessed 2026-10-23; experimental version: DuckDB 1.1.3).

Minutes of the 1st Project Meeting

Date Sunday,June 7, 2026

Time 2:30 pm

Place Room 3523

Present Prof Ke Yi, Chai Guanpeng

Apology None

Note-taker Chai Guanpeng

1.Discussion items

The detail of the project and the demands of the proposal

2.Meeting adjournment and next meeting

The meeting adjourned at 3:00PM. The next meeting will be held in June

Minutes of the 2nd Project Meeting

Date Wednesday, June 10, 2026

Time 4:00 pm

Place Room 3520

Present Prof Ke Yi, Chai Guanpeng

Apology None

Note-taker Chai Guanpeng

1.Discussion items

The detailed requirements of the project

2.Meeting adjournment and next meeting

The meeting adjourned at 4:30, The next meeting will be held in July

Minutes of the 3rd Project Meeting

Date Monday, July 2, 2026

Time 2:30 pm

Place Room 3523

Present Prof Ke Yi, Chai Guanpeng

Apology None

Note-taker Chai Guanpeng

1.Discussion items

Reporting the progress of the project

2.Meeting adjournment and next meeting

The meeting adjourned at 3:00PM. The next meeting will be held in August

Minutes of the 4th Project Meeting

Date Thursday, August 6, 2026

Time 3:30 pm

Place Room 3523

Present Prof Ke Yi, Chai Guanpeng

Apology None

Note-taker Chai Guanpeng

1.Discussion items

Reporting the final result of the project and starting working on the report

2.Meeting adjournment and next meeting

The meeting adjourned at 4:00PM.